Lab 3: Component Modelling & Architectural Pattern Selection

Course: Software Engineering Lab (UE23CS352A) - Department of CSE, PES University

Student: Navaneeth Tanuboddi | SRN: PES1UG24CS709 | Semester: 5th Sem

Assigned Topic: Problem Statement #45 - Multi-Cloud Asset Cost Optimization Portal (MCCOP)

1. Laboratory Objectives & Learning Outcomes

* Evaluate canonical architectural patterns (Layered, Microservices, Client-Server) across scalability, coupling, and fault isolation.
* Select and justify the optimal architectural style for both the assigned project (MCCOP) and the benchmark scenario (Coffee Kiosk).
* Construct complete UML 2.0 Component Diagrams with provided (ball) and required (socket) interfaces, ports, and assembly connectors.

2. Comparative Evaluation of Architectural Patterns

Metric	Layered (N-Tier)	Microservices	Client-Server
Topology	Horizontal concentric tiers (Presentation, Business, Data)	Autonomous, domain-bounded services via network APIs	Centralized server servicing lightweight client nodes
Coupling	High between adjacent layers; tight compile-time bounds	Loose coupling via versioned REST/gRPC contracts	Tight client-to-server request/response protocol binding
Scalability	Monolithic; entire stack must scale horizontally	Fine-grained per-service independent horizontal scaling	Vertical server scaling or load-balanced server pooling
Fault Tolerance	Low; exceptions in one tier can crash shared process	High; failures in non-critical services are isolated	Single Point of Failure (SPOF) if server node crashes
Operational Effort	Low; single deployment artifact and centralized logs	High; requires container orchestration (K8s) & tracing	Moderate; server cluster management & DB pooling
Network Latency	Negligible; in-memory function calls & pointers	High; network serialization & multi-hop RPC latencies	Low to moderate; standard client-to-server TCP/TLS
Data Consistency	Strong consistency (ACID) in single relational DB	Eventual consistency; saga patterns & distributed events	Strong consistency managed centrally at server database
Scenario Fit	Kiosks, embedded systems, localized desktop tools	High-throughput cloud analytics & multi-cloud FinOps	Internal admin consoles, basic request/response apps

3. Multi-Cloud Domain Analysis: Why Microservices is Selected

Processing multi-cloud billing involves massive batch CSV/Parquet uploads (AWS CUR up to 5 GB) alongside concurrent interactive querying of analytical dashboards. A Layered Monolith would encounter thread starvation and latency degradation, while Client-Server creates a fragile SPOF. Microservices enables independent horizontal worker scaling and complete fault isolation.

4. Assigned System: Multi-Cloud Asset Cost Optimization Portal (MCCOP)

Architectural Choice: Microservices / Service-Oriented Architecture with Ball-and-Socket Interface Model

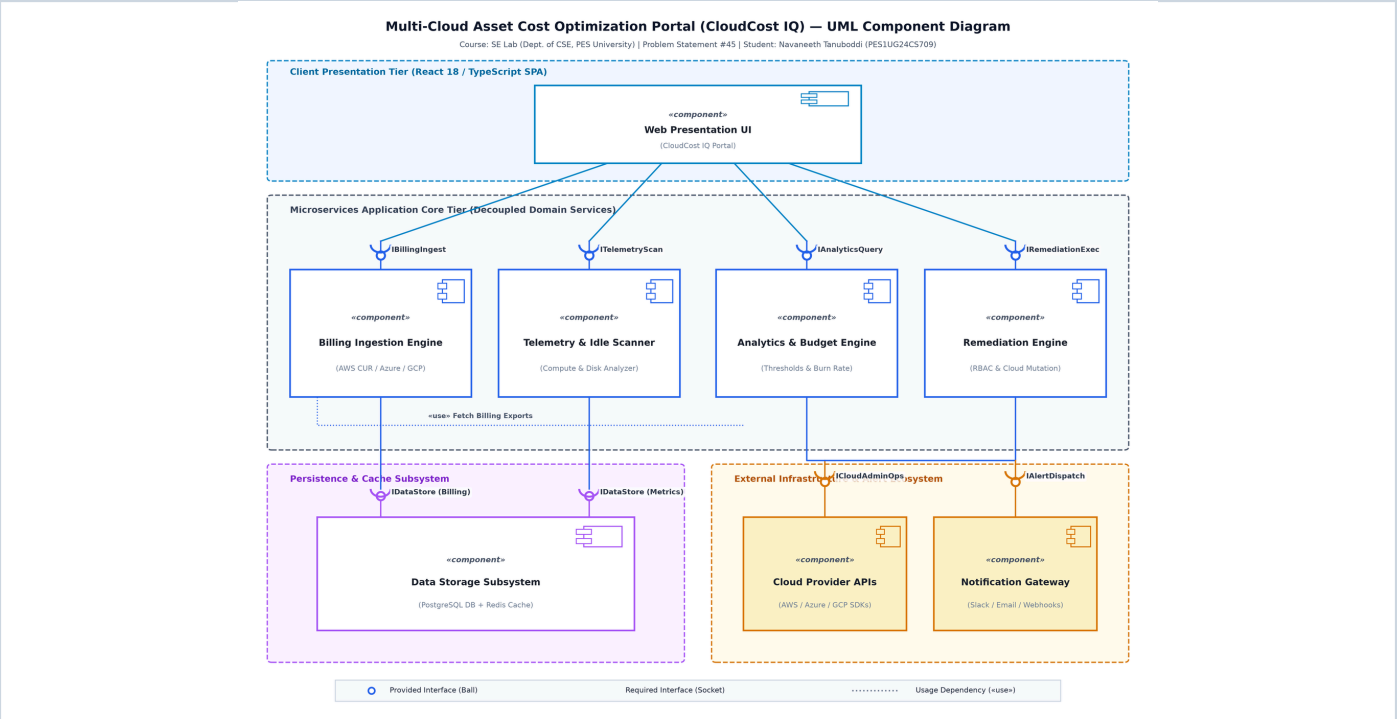


Figure 1: UML Component Diagram for Multi-Cloud Cost Optimization Portal (CloudCost IQ)

Interface Specifications (Ball-and-Socket Model)

Interface	Provided By (Ball)	Required By (Socket)	Protocol	Payload & Scope Description
IBillingIngest	Billing Ingestion Engine	Web Presentation UI	REST / HTTPS	Ingests AWS/Azure/GCP billing CSVs; returns normalization stats
ITelemetryScan	Telemetry & Idle Scanner	Web Presentation UI	REST / HTTPS	Queries detected idle compute instances & unattached block stora
IAnalyticsQuery	Analytics & Budget Engine	Web Presentation UI	REST / HTTPS	Aggregates multi-cloud spend, burn rates, and budget forecasts
IRemediationExec	Remediation Engine	Web Presentation UI	HTTPS / MFA	Dispatches approved remediation actions with mandatory audit log
IDataStore	Data Storage Subsystem	All Application Engines	TCP / JDBC	ACID persistence for billing rows, telemetry cache, and audit histor
ICloudAdminOps	Cloud Provider APIs	Remediation Engine	HTTPS / SDKs	Executes cloud mutations (ec2:StopInstances, ebs:DeleteVolume)
IAlertDispatch	Alert Gateway	Analytics & Budget Engine	Webhook / JSON	Sends multi-channel alerts (Slack, Email, PagerDuty) on budget br

Technical Justification Summary:

* Security: Cloud provider credentials (IAM roles, service account tokens) are isolated strictly inside private worker runtimes.
* Performance: Asynchronous streaming ingestion satisfies NFR-001 (<= 2 min processing for 5 GB datasets) without UI blocking.

5. Benchmark Scenario: Self-Service Coffee Kiosk System

Lab Manual Benchmark: 5 Components across Presentation, Business, and Data Layers with Ball-and-Socket Notation

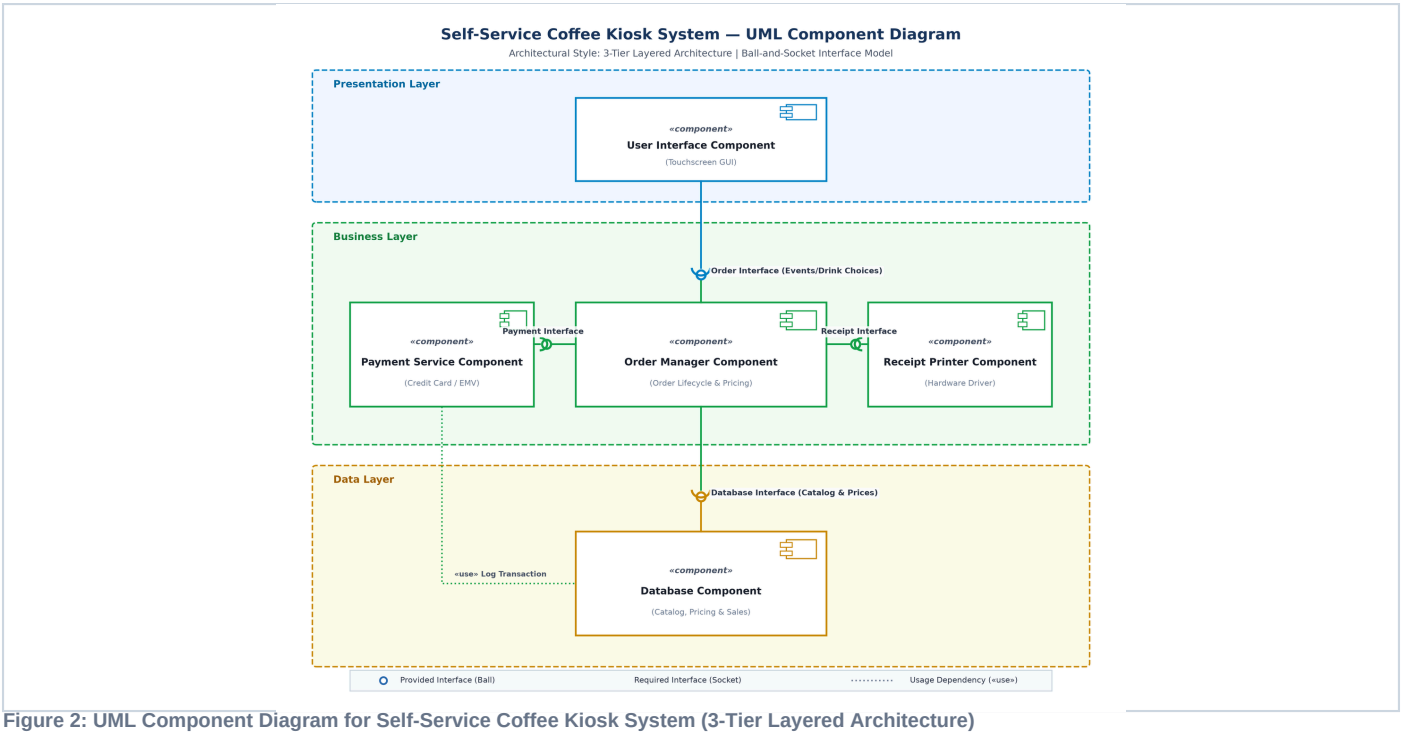


Figure 2: UML Component Diagram for Self-Service Coffee Kiosk System (3-Tier Layered Architecture)

Coffee Kiosk Component Identification & Interface Mapping

Component Name	Layer	Provided Interface	Required Interface	Responsibilities
User Interface Component	Presentation	None (Client GUI)	Order Interface	Captures touchscreen drink selections (Espresso, American
Order Manager Component	Business	Order Interface	Payment, Receipt, DB	Calculates totals, manages order lifecycle state, coordinates
Payment Service Component	Business	Payment Interface	Database Interface	Communicates with EMV card reader, validates charges, log
Receipt Printer Component	Business	Receipt Interface	None (Driver Endpoint)	Formats order receipt and drives physical thermal printer ha
Database Component	Data	Database Interface	None (Data Store)	Stores coffee recipes, drink size pricing rules, taxes, and tra

Key Architectural Characteristics:

* Clean Separation of Concerns: Hardware drivers (printer) and screen GUI are isolated from ordering business logic.
* Embedded In-Memory Execution: All components reside on local kiosk hardware, eliminating network dependencies.

6. Written Justification Document (Per Lab Manual Step 5 Requirements)

Architecture Selection: "We chose Layered Architecture for the Self-Service Coffee Kiosk System."

1. Architectural Choice:

We selected the 3-Tier Layered Architectural Pattern, organized into Presentation, Business, and Data layers.

2. Two Specific Scenario-Related Reasons:

a) Separation of UI and Hardware Peripherals: The kiosk integrates physical touchscreen hardware and a thermal receipt printer. The layered model isolates peripheral driver protocols from business rules, allowing printer hardware upgrades or touchscreen UI redesigns without altering pricing calculations or order state logic.

b) Self-Contained Deployment on Standalone Kiosk Hardware: The system functions as a localized physical appliance in a cafe. A distributed microservices architecture would create unnecessary network latency and deployment overhead. The layered architecture provides a deterministic, single-process execution environment.

3. Security Advantage (PCI-DSS Cardholder Data Isolation):

By isolating credit card processing into a dedicated Payment Service Component within the business layer, cardholder data and EMV terminal communications are segregated from the UI layer. The touchscreen GUI never stores or accesses primary account numbers (PAN), minimizing the PCI-DSS compliance audit scope and shielding financial data.

4. Performance Benefit (Zero-Network In-Memory Latency):

Since all layers execute locally within a shared hardware runtime, communication between UI, Order Manager, and local database utilizes high-speed in-memory method invocations. This ensures sub-millisecond touchscreen response times and sub-second receipt printing upon payment completion.

7. Traceability Matrix: Requirements to Components & Code Implementation

Requirement	Scope Description	Mapped Component	Interface	Source Code Implementation
FR-001	Multi-Cloud Billing Ingestion & Tagging	Billing Ingestion Engine	IBillingIngest	project/src/services/billingParser.ts
FR-002	Idle Compute Instance Detection (<5% CPU)	Telemetry & Idle Scanner	ITelemetryScan	project/src/services/idleDetector.ts
FR-003	Unattached Block Storage Identification	Telemetry & Idle Scanner	ITelemetryScan	project/src/services/storageAnalyzer.ts
FR-004	Budget Monitoring & Alert Dispatch	Analytics & Budget Engine	IAnalyticsQuery	project/src/services/budgetEngine.ts
FR-005	Automated Remediation & Audit Logging	Remediation Engine	IRemediationExec	project/src/services/remediationEngine.ts
NFR-001	Ingestion Performance & Scalability (<=2 min)	Billing Ingestion Engine	Async Ingestion	project/src/services/billingParser.ts
NFR-002	Security & Role-Based Access Control (RBAC)	Remediation Engine	MFA Gate	project/src/components/RemediationHub.tsx